

MindTrip 面试问题全集

100题项目实答：原题55道 + Agent追问35道 + 指标追问10道 · 2026年9月8日

我做这个项目，重点是把旅行需求变成可检查的行程：用检索找证据，用结构化输出承载日程，再把时间、费用、来源和任务状态分别处理。最有收获的部分，是发现“返回成功”和“用户任务完成”之间其实隔着很多问题。

这份问答保留原题编号和P0/P1顺序，可按第一人称练习。回答中的历史实验与当前功能分开说明；未完成的能力不算已交付，也不把本地验证说成生产经验。原始问题PDF另存为同目录备份。

先讲清楚的数字口径

证据层	可引用的结果与边界
351条检索	历史BM25: Recall@5 0.9801, MRR@10 0.9204, HTTP检索P95约31ms。不是Dense或Reranker成绩。
70条离线规划	Base分支: 首次3/70 → 28/70; 约束宏平均78.76% → 84.33%; 最终18/70 → 27/70。合并模型另有一组结果，不混用。
150条检索对照	Dense与同一Top20重排: MRR 0.977778 → 0.984444, Recall不变; 进程内P95约44.13 → 313.10ms。
200条真实HTTP	HTTP200为200/200, 最终业务成功32/200且均为澄清; 130条规划零成功。修复恢复2/170。全请求P95约105.195秒。
BGE历史性能	RTX 4080, batch=1, seq=32/128/512; TRT FP16主干混合精度平均3.39—5.04ms。相对ORT FP32约4.87—7.35倍, 存在profiling与I/O口径差异。

原题第19题的“待补对照”已有后续150条实验，但不能替代351条旧集。第44题的倍率对应ORT FP32; FP16基线会得到另一组倍率。详见对应回答。

一 评测与证据链

P0 1. 任何项目指标都要先说清哪四件事？——测试集、指标定义、Baseline、失败案例。

我会先交代测试集是什么、指标怎么算、跟哪个基线比，再拿一个失败案例说明这个数字的边界。比如我讲检索 Recall@5=98.01%，会补上这是351条查询上的 BM25 结果，不能接着把它说成 BGE-M3 加 Reranker 的效果。否则数字虽然是真的，结论也可能是错的。

这个项目给我的一个教训是，测试跑完和问题解决是两回事。200条请求全部返回 HTTP200，但最终只有32条达到业务要求。先把分母和成功条件讲明白，比拿一个很漂亮的百分比更重要。

P0 2. MindTrip 目前有哪些互相独立的评测基线？为什么不能混用？

我会分层讲。351条是历史 BM25 检索回归；70条是固定任务和检查器的离线规划对照；200条走本机真实 HTTP Agent；TensorRT 那组只测 BGE 模型执行路径。后来还补了150条 Dense/Reranker 同集对照，以及单独的回答质量检查。

它们的输入、模型分支、计时范围和成功标准都不一样。检索命中文档，不代表模型能生成合法行程；embedding 变快，也不代表整条请求按同样比例加速。我不会把这些数字拼成一个“系统准确率”。

P0 3. 测试集的数量、任务类型、数据来源和审核方式分别怎么说明？

351条主要是语料锚定的检索回归查询，模板里包含较多景点名称。70条覆盖57个城市，是从语料派生、用程序检查的规划任务，不是人工黄金集。200条 HTTP 数据由70条原规划、60条多条件变体、30条缺信息、30条极端或冲突、10条不确定性请求组成，并不是200个独立人工案例。

后来的150条检索对照分为100条知识查询和50条多条件查询。部分资产有后续用户审核声明，但声明只覆盖列明的文件和哈希，不能顺手扩展到70条规划或新200条数据。面试时我会把“生成、程序校验、AI审核、人工看过”分别说清楚。

P0 4. Ground Truth 怎么构建，为什么不能让同一个 LLM 自己出题再自己打分？

我会先从原始资料定义可接受的答案和证据，再写查询，最后让另一轮审核检查两者是否真的对应。检索题至少有相关文档ID；规划题还要有预算、日期、禁止项等预先冻结的检查条件。标注里不确定的地方，我宁可保留争议，也不临时改成让当前模型通过的答案。

同一个 LLM 出题再评分，很容易把自己的表达习惯当标准，或者一起接受同一个错误。换一个模型能减少部分相关偏差，但也不等于人工真值。项目里就发现过把0.9元电子导览当成门票、把闭园时间当成最晚入园时间的标注问题，这些必须回到原文判断。

P0 5. Task Success 为什么必须是可程序化、可重复判定的？

因为我希望下次换提示词、换运行环境，仍然能对同一份结果得到同一个判断。天数是不是三天、预算有没有超、活动是否重叠，这些有明确输入和输出，应该由代码检查，不能取决于评分模型这次心情怎么样。

当然，程序也只能判断写进去的规则。景点是否真的适合老人、解释是否有帮助，不能硬凑成几个字符串匹配就说评估完了。我的做法是把确定性成功条件和需要语义审核的条件拆开，并保留检查器版本。

P1 6. 如果一个指标很高，但说不清分母、标注方式和失败样例，为什么可信度仍然低？

分母决定这个高分在回答什么问题。假如只统计成功生成的31条回答，把19条没有可评分事实的回答放到一边，均分看着不错，却没有说明剩下的用户拿到了什么。标注不完整也会让漏召回看起来像命中了全部相关文档。

我更愿意展示一个略低但能复算的数字：给出样本、预测、判断规则和失败记录。面试官拿其中一条追问，我能解释为什么判对、哪里还没覆盖，这个成绩才站得住。

P1 7. 优化前后对比时，哪些变量必须固定，才能证明提升来自方案本身？

至少要固定测试集与语料版本、标注、模型权重、输入证据、检查器、生成参数和输出预算。性能对比还要固定硬件、batch、长度、预热和计时边界。只研究 Reranker 时，就应让它和基线共用同一批候选，不能同时换查询改写或扩大召回。

历史70条 Base 对照固定了任务、检查标准和1400-token上限，调整的是一组提示词要求。因此我能说整组改动与指标变化对应，不能进一步声称某一句提示词单独贡献了多少。要讲到这个程度，还得做更细的消融。

P1 8. 至少准备哪两类真实失败案例，面试时应该按什么逻辑复盘？

我会准备一个“输出看起来像样，但契约或业务失败”的案例，再准备一个“优化没救回来，甚至发生回归”的案例。项目里有活动引用不合法、结构化结果不完整的问题；70条对照里也有改动前修复后成功、改动后反而失败的记录。

复盘时先给原始需求和预期，再看实际输出、失败检查、定位证据、采取的改动和复测结果。工程上还有一个很具体的例子：任务被取消了，进度却先发出了 completed。我把终态决策和事件生成放到同一处锁内处理，再验证取消和完成竞争时只有一个最终状态。只讲“加了重试”解释不了这种问题。

二 RAG检索指标与Reranker

P0 9. 为什么 RAG Retriever 不直接看 Accuracy，而优先看 Recall@K、MRR 等排序指标？

检索返回的是一个有顺序的候选列表，不是给每篇文档做独立二分类。如果全库有上千篇，绝大多数都不相关，把它们全部判成不相关也能得到很高的普通 Accuracy，但用户需要的那篇可能一篇都没找回来。

我关心的是：有用证据有没有进前几名，最先出现在哪个位置。所以会优先看 Recall@K 和 MRR，再结合延迟。这和后面模型是否回答正确是不同的问题。

P0 10. Recall@5 是什么，MindTrip 的 Recall@5 应该怎样计算？

对一条查询，我拿返回的前5个文档ID，与预先标注的相关ID集合求交集，再除以这条查询标注的相关文档数；最后对所有查询取平均。项目代码里对应的是 `expected_source_ids`，概念上就是题目中的 `relevant_doc_ids`。

比如有两个相关文档，前5只找回一个，Recall@5就是0.5，不是1。如果每题恰好只有一个相关ID，Recall才会退化成“前5是否命中”的比例。351条历史 BM25 结果是0.9801，我会同时说明标注完整性仍有限，不能把模板命中能力说成复杂旅行问题都能解决。

P0 11. Recall@K 和 Precision@K 有什么区别？为什么 RAG 第一阶段通常更怕漏召回？

Recall@K看“应该找回的证据找回了多少”，Precision@K看“交给后续的K条里面有多少是相关的”。同样返回5篇，其中1篇相关，如果这题只有1篇相关文档，Recall是1，而Precision是0.2。

第一阶段通常更怕漏掉关键证据，因为 Reranker 只能调整已经召回的候选，不能把没进候选集的文档变出来。但这不代表候选越多越好：太多无关材料会增加重排成本，也会干扰生成。所以先保召回，再控制进入上下文的质量和数量。

P0 12. MRR@10 是什么？为什么它适合体现第一个相关结果是否排得足够靠前？

我会看每条查询前10名里第一个相关文档的排名。第1名计1，第2名计1/2，第5名计1/5；前10都没有就计0，最后取平均。这就是 MRR@10。

它适合看用户或模型要往下读多久才能碰到第一条有用证据。它也有局限：第一篇已经排第1，第二篇关键补充材料有没有找回来，MRR不太敏感，所以我不会用它代替 Recall。

P0 13. 为什么 MindTrip 优先选择 Recall@5 + MRR@10，而不是堆很多检索指标？

这两个指标正好回答我最关心的两个问题：证据是否进入有限上下文，以及最有用的材料是否靠前。固定 K 后，结果比较容易复算，也方便用具体查询解释一次优化到底改变了什么。

我不会为了显得完整把所有排序指标都写进简历。真要展示改进，我宁愿给一条前后排名和延迟变化。后来的 Reranker 对照就只有两条查询的首次相关排名改善，这比只报一个均分上涨更有信息量。

P0 14. 为什么需要 Reranker？请讲清 Top-N Retrieval -> Rerank -> Top-K Context。

我会先用成本比较低的检索器找出 Top-N 候选，再让 Reranker 对查询和每篇候选一起打分，最后取 Top-K 放进生成上下文。第一步负责大范围筛选，第二步负责在小范围里辨别细微相关性。

项目后来的离线对照就是 Dense 先取20篇，重排仍然只用这20篇。结果不是想象中那么明显：Recall@5 没变，MRR只小幅上升，延迟却增加不少。所以我的结论是有接入价值，但这批数据不足以支持默认给所有请求开启。

P0 15. Dense Retrieval 和 Reranker 分别解决什么问题？为什么不直接让 Reranker 扫全库？

Dense 会把查询和文档分别编码成向量，文档向量可以预先算好，所以在线找候选比较高效。Reranker 则把查询和候选放在一起判断，能更细地看它们是否真正匹配，但每个查询都要重新处理这些配对。

如果直接重排全库，成本会随着文档数一起增长，长文本尤其明显。我会先检查目标证据是否被召回；没进 Top-N，先改召回。已经在候选里但位置不好，再看重排。这样定位问题，不容易花钱在错误的环节上。

P0 16. 如何公平比较 Dense-only 与 +Reranker？Baseline 应该怎么设计？

我会冻结同一组 query、相关ID、语料和 embedding，先保存 Dense 的 Top20；实验组只对这20条重排，基线直接使用原排序。两边都算 Recall@5、MRR@10，也记录相同边界的耗时，不能只展示效果。

项目已有150条这样的对照：MRR@10从0.977778到0.984444，Recall@5均为0.990667。进程内预热后的P95从约44.13ms增到313.10ms。这说明在这一批数据上重排收益很小，不能直接推广到所有查询，更不能拿旧351条 BM25 成绩当这次 Dense 基线。

P1 17. 为什么相关文档从第 5 名被推到第 1-2 名时，MRR 会明显提高？

因为倒数对前几名的变化比较敏感。相关文档从第5名到第1名，这条查询的贡献从0.2变成1；到第2名则变成0.5。只要它们都在前5，Recall@5可能完全不变，但 MRR能把排序改善反映出来。

总体提升还取决于发生了多少条。项目150条里只有两条从第2名到第1名，总增量就是2乘以0.5再除以150，约0.006667。这也解释了为什么不能把“个别排序变好”说成全局有显著改善。

P1 18. Precision@5 为什么可以内部观察，但不一定写进简历？

Precision@5内部看很有用，可以发现候选里混了多少无关材料。但如果标注只记录了一篇锚定文档，其他实际有帮助的文档没有被标进去，Precision可能被低估。直接写进简历，往往还要花不少时间解释标注覆盖。

我会优先展示能说明任务目标、证据比较完整的指标。不是 Precision不重要，而是一个指标能不能公开使用，取决于它的定义和标注是否撑得住，而不只是计算脚本能不能跑。

P1 19. 当前 MRR@10=0.9204 为什么不能直接写成“加入 Reranker 后提升 xx%”？[待补同集对照]

0.9204来自351条历史 BM25 基准，当时没有在这同一集合上做 Dense-only 和加重排的配对实验，所以它本身不能证明 Reranker 提升了多少。

这里原题需要更新：项目后来已有另一组150条同集对照，MRR从0.977778到0.984444，绝对增量约0.006667，相对约0.6818%，Recall不变。我可以讲这组结果，但不能把0.9204拿来当它的起点。两条查询改善、延迟增加，才是这次实验完整的结论。

P1 20. 351 条检索集的 relevant_doc_ids 应该怎样标注和人工审核？[待补完整标注证据]

我会先定义“相关”是能支持回答这个查询，而不是文档里碰巧出现景点名。然后为每条 query 保存相关文档ID、支持片段和判定理由；从不同检索器收集候选，再由人工核对漏标、歧义和多篇共同支持的情况，最后冻结版本。

351条旧集目前不能声称已经补齐逐题穷尽标注。后来的150条用了锚定文档加 lexical Top5 的候选池，也仍可能漏标。我不会根据模型当前召回了什么，就把那些文档补成“正确答案”，否则评价目标会跟着模型一起漂移。

三 Ground Truth与生成质量

P0 21. 检索评测的 Ground Truth 至少为什么要有 query + relevant_doc_ids?

query定义信息需求，relevant_doc_ids定义哪些文档能满足这个需求。只有问题没有相关ID，就没法确定一个排序结果有没有找对；只有参考答案没有ID，也很难判断到底是哪一步漏了证据。

ID还必须能回到固定语料版本中的原文。项目这轮区分了父级 source_id 和 chunk_id，就是为了防止切分后找不到原始来源。多个块来自同一篇文档时，评分还要统一粒度，不能靠重复返回同源块把命中数刷高。

P0 22. 如果评估最终生成结果，还应准备哪些参考答案或 supporting evidence?

我会补参考答案要点、支持这些要点的原文片段、出处和时间；对规划结果，再补硬约束、可接受变体和禁止推断。旅行没有唯一行程，所以不能简单要求生成内容和一份标准攻略逐字一致。

比如预算题要先明确是全员总预算还是人均预算，门票和导览也不能混在一起。开放时间、票价没有证据时，正确结果可能是说明未知并请用户确认，而不是为了凑齐答案写一个看起来合理的数字。

P0 23. Faithfulness 是什么？它和 Correctness 有什么区别？

Faithfulness检查回答里的事实陈述是否得到提供的上下文支持；Correctness则看这些陈述是否符合可信参考或实际事实。一个回答完全照着过期资料说，Faithfulness可以很好，但现实中照样是错的。

项目里最容易混淆的就是引用。以前只要 source_id 存在就会标 verified，实际上这只证明“有这个来源”。现在改成单独标来源是否找到，具体事实仍待确认。引用存在、引用相关、字段有证据和事实正确，不能合成一个布尔值。

P0 24. Answer Relevancy 是什么？为什么它和 Faithfulness 二选一即可？

Answer Relevancy看回答有没有正面解决用户问的问题，是否遗漏重点或夹杂大量无关内容。一个回答可以每句都有证据，却完全没回答“带老人去是否合适”，这时支持度高、相关性仍然差。

题目说二选一，我理解是简历展示要克制，不是两个指标在技术上等价。项目后来已有相关性补充：50条回答的自定义0—4分量表归一化均值是0.685，不是68.5%的正确率。已有 Faithfulness 只有31条可评分、覆盖62%，两组口径也不能混成一个总分。

P1 25. 为什么“回答有上下文支持”不等于“现实世界事实绝对正确”？

因为上下文也会过时、写错，或者根本不支持用户真正问的那个条件。旅游资料里“营业到22:30”不必然等于22:30仍然允许入园；文档写了可看日出，也不代表当天开放时间允许进去。

我会把来源时间、支持片段和不确定性一起保留。对于价格、库存、实时营业这些容易变的事实，优先用能覆盖出行日期的可信工具或官方信息；没有就明确待确认，不让“检索到了”替我背书。

P1 26. 为什么求职项目不建议同时堆 Precision、Recall、MRR、nDCG、Faithfulness、Correctness、Relevancy 等一长串指标？

面试时间有限，每个指标都应该对应一个我要解释的问题。指标堆得多，既增加口径混用的风险，也容易把模型、检索、性能和业务结果混在一起。最后反而说不清最重要的瓶颈是什么。

我会用少量主指标讲清检索质量、任务成功和耗时，再拿失败案例支撑。其他指标留在内部诊断。例如200条 HTTP 的主要问题是结构化生成失败，这时候继续加很多检索指标，并不能解释为什么行程根本没生成出来。

四 BGE M3 FAISS与结构化Agent

P0 27. 为什么选择 BGE-M3？要从中文/多语言、检索能力、开源可部署、本地优化哪些角度回答？

我选 BGE-M3，主要因为项目以中文旅行资料为主，同时可能遇到英文地点和跨语言表达，需要一个能本地部署的多语言 embedding 模型。它也方便固定权重、复用索引，后续做 ONNX 和 TensorRT 路径的性能、数值一致性检查。

不过，模型支持什么不等于项目全都用了。我们的 Dense 对照使用的是它的向量表示；线上混合检索里的 BM25 是单独的词法通道，不能说已经把 BGE-M3 的各种检索模式都集成好了。当前我更愿意保持模型不变，先解决切分、过滤和证据语义的问题。

P0 28. 为什么当前选择 FAISS？什么时候会考虑换成 pgvector / Qdrant / Milvus？

当前语料规模和本地演示目标下，FAISS 的部署负担小，向量检索路径也比较直接。我不需要一开始就运维一套复杂的向量数据库，先把查询、索引版本和结果契约做好更实际。

如果后面需要频繁增删、持久化事务和业务表联合过滤，我会考虑 pgvector；需要独立服务、向量与元数据过滤能力，再评估 Qdrant 或 Milvus。迁移依据是访问模式和运维需求，不是哪个名字更像生产系统。

P0 29. Pydantic 能解决 LLM 幻觉吗？它真正保证的是什么？

不能。Pydantic 能验证结构、类型以及我显式定义的约束，比如天数范围、预算必须是有限数、时间字段格式，但它不知道某个景点今天是否开放，也不知道模型编的票价是真还是假。

这个项目就遇到过空白目的地和 Infinity 预算的问题：有字段、有数值类型，还不够，必须补非空白和有限数校验。即使这些都通过了，也只能说输入输出符合契约，事实和业务可行性还需要后面的检查。

P0 30. 事实正确性为什么仍需要 RAG、业务规则、证据追踪与评测？

这几层负责的问题不同。RAG给模型可引用的材料，规则检查时间、费用、天数等确定性关系，证据追踪让我能回到原文，评测则观察这些环节在真实任务里究竟有没有一起发挥作用。

我不会承诺这些加起来就能消灭幻觉。比如模型给了一个合法引用，却用它支撑不相干的价格，Schema和ID检查都可能通过。因此我把“来源找到”和“事实核验”拆开；缺少字段级证据的活动仍然提示确认。

P0 31. LangGraph 相比普通顺序 Chain 的价值是什么？为什么 MindTrip 需要 state、branch、validate、repair/retry？

如果只有固定的两三步，顺序 Chain 完全能做。我用 LangGraph，是因为规划需要携带请求、来源、工具结果和告警状态，还涉及节点恢复、条件入口以及后续校验。把节点职责写出来，比把所有异常和条件塞进一个函数容易定位。

不过这里有个边界：现有 Job/MySQL 才是任务和审批状态的权威，LangGraph负责节点执行；另一个 HTTP Agent 的两次候选修复是有界编排，尚未并入同一条持久化链路。我不会把它们讲成已经实现了完整原生 checkpointer 或任意节点回放。

P0 32. retrieve -> plan -> validate / repair 各阶段分别解决什么问题？

retrieve负责取证据，项目中间还有天气和酒店工具节点；plan把需求与这些数据转成 TripPlan；validate重新检查引用、时间和费用关系。可安全归一化的内容，例如费用加总，可以确定性处理；需要模型重写的错误才考虑有限次数的 repair。

retry和repair也得分开：上游网络瞬断是传输重试，JSON不合契约是输出修复。它们叠加后都会消耗时间和模型调用，所以我增加了任务级调用预算与取消检查，不能让每层都各自重试三次，最后变成用户不知道的十几次调用。

P1 33. FAISS 在当前规模下的优势是什么？规模扩大、服务化、多租户或复杂过滤时迁移要考虑什么？

当前优势是简单、可在本机运行，而且固定索引容易复现。真正扩大规模后，我会先看向量数、维度、内存、更新频率和查询延迟，再决定索引类型或服务迁移，而不是仅凭文档数量选产品。

多租户和复杂过滤尤其不能只靠前端传一个过滤字段。我需要服务端身份边界、过滤后补召回、稳定ID和删除语义。迁移时还要统一 embedding 版本、归一化和距离度量，保留旧索引回退路径，再用固定查询核对结果，而不是新服务启动了就算迁移完成。

P1 34. 为什么结构化输出有效不等于旅行规划业务一定正确？

合法 JSON 只说明能解析，Schema通过也不代表行程符合需求。模型可以给出格式完整的四天行程，但用户只要三天；也可以让两个活动时间重叠，或者把人均费用当成全员费用。

还有更隐蔽的一种：未知票价写0，数值预算检查通过了，却不能证明真实可负担。这类情况我会保留待确认，不把“通过格式门禁”宣传成“旅行规划成功”。70条和200条评测都说明，这两者之间差距很大。

五 约束满足率与端到端成功率

P0 35. Constraint Satisfaction Rate 是什么？城市、预算、天数、兴趣、禁止项、时间约束如何检查？

我把每条任务适用的约束列成独立检查项，再计算通过比例。城市 and 天数可以直接对结构化字段；预算要按人数和费用口径重算；时间检查正向时长、重叠和指定窗口；禁止项则用明确名称或规则核对。

兴趣匹配和自由表达不能都靠关键词保证。目前支持的是有限规则，不是通用自然语言约束求解器。报告还要说明是先对每个任务算比例再平均，还是把所有检查项合起来算。历史70条报的是约束宏平均，不能和其他集合的微平均直接比较。

P0 36. End-to-End Task Success Rate 怎么定义？一个 TripPlan 什么条件下才算整体成功？

对于规划任务，我会要求输出能解析、符合 TripPlan 契约，目标城市 and 天数正确，所有必需的硬约束都通过，且有实际活动与可追溯引用，才记一次整体成功。中途报错、缺结果或关键条件失败都保留在分母里。

对于缺信息的任务，正确澄清本身可以是预期行为，但这不等于已完成旅行规划。200条 HTTP 的32次最终成功全部是澄清类结果，130条规划任务没有成功。这一点必须主动讲，不能只说“端到端成功率16%”让人误解成规划能力。

P0 37. Constraint Satisfaction 和 Task Success 有什么区别？

约束满足率看的是局部条件通过了多少；Task Success看这一个任务是否满足全部必要条件。十项检查过九项，约束满足率可以是90%，但如果唯一没过的是预算，任务仍应失败。

所以历史70条里，约束宏平均到84.33%，最终 E2E仍只有38.57%，两者并不矛盾。这个差距说明有些局部问题改善了，但成功交付整份行程仍有短板。我会同时看两者，不用局部高分掩盖整体失败。

P0 38. 为什么不能只靠 LLM Judge 主观判断规划是否成功？

LLM Judge会受措辞、顺序和自身偏好影响，也可能被一段很顺畅的解释说服。预算超支、时间重叠这类问题，代码可以精确判断，没必要再让模型猜一次。

语义问题确实可以用 Judge辅助，但要固定量表、做控制样本、保存原始评分并抽查偏差。项目的回答相关性补充里就出现过额外要求和局部矛盾的评分，我保留了这些记录，没有因为分数不顺眼就手工改高。

P1 39. “预算不超过 2000” “第二天下午留 3 小时” “不去游乐场” 分别怎样做确定性校验？

预算不超过2000，我会先确认总预算还是人均，再按活动的 group/per-person 口径计算全员费用。第二天下午留三小时，需要先把“下午”落成明确区间，例如13点到18点，再计算活动和交通占用的区间并集，看有没有连续三小时空档。

“不去游乐场”要用明确的地点类别或可靠标签判断，不能只看名字里有没有“乐园”。目前系统并没有通用地理解所有自由表达；遇到窗口或分类不清楚，我会追问，而不是偷偷选一个解释后宣称约束通过。这部分是完善规则的方法，不代表所有语义都已实现。

P1 40. 70 条离线规划集 Before/After 的三个指标分别改善了什么？

这组数字属于历史 Qwen2.5-3B-Instruct Base 分支。首次业务成功从3/70到28/70，说明不用后续修复就成功的任务增加；约束宏平均从78.76%到84.33%，说明局部检查整体改善；最终 E2E从18/70到27/70，说明最后真正完成任务多了9条。

改动主要是明确单一 JSON 契约、减少完整证据回显、写清引用和人数预算规则，没有新训练。也要主动补一句：合并模型另一组 Final E2E是8/70到12/70，约束宏平均反而下降。不能拿 Base 的好成绩替合并模型背书。

P1 41. 为什么 70 条规划集必须说明 corpus-derived / programmatic-verified，而不能说成人工黄金集？

因为这些任务来自已有语料和程序化构造，部分断言能被脚本重复检查，但并不意味着每条需求、相关文档和现实事实都经过独立人工完整标注。语料派生还可能让问题措辞更贴近材料，降低难度。

我会直接说“70条语料派生、程序化检查的离线规划集，覆盖57个城市”。这已经能说明我做了固定对照。没必要把它包装成人工黄金集；一旦被问到标注人员、争议处理和数据独立性，就解释不通了。

六 TensorRT Benchmark与FP16选择

P0 42. 3.39-5.04 ms 到底是什么延迟？为什么绝不能说成“整个 RAG 只有 3 ms”？

3.39到5.04ms是历史 RTX 4080 上，BGE-M3 TensorRT FP16主干混合精度路径在不同序列长度下的平均执行耗时。它不是P95，也不包含整个RAG请求。

用户一次请求还有分词、查询编码之外的检索与过滤、可能的重排、网络往返和大模型生成。历史计时器本身还包含同步和输出拷回CPU，因此连“纯GPU kernel只有3ms”也不准确。我会明确说这是哪个runner 的平均耗时，不把局部优化说成整条链路都这么快。

P0 43. 这个 Benchmark 测的是哪个模型、哪一段链路、什么 GPU、batch 和 sequence length？

模型是 BGE-M3，硬件是 RTX 4080，batch固定为1，序列长度分别测32、128、512，先预热10次，再测100次。对应 TensorRT 路径的平均耗时约3.3867、3.8543、5.0394ms。

这不是 Qwen 的生成速度，也不是150条检索对照的执行路径。后者用的是 transformers 的编码方式。面试前我会把实验目录和 CSV一起准备好，避免把模型名称、运行时和测试任务混成一组。

P0 44. 4.9-7.4x 加速的 Baseline 是什么？为什么不同 seq length 的加速倍数不同？

原题里的约7.35、6.27、4.87倍，对应同份1852-bgepipe历史记录 ORT CUDA FP32基线：约24.9011、24.1655、24.5393ms，分别除以 TensorRT 的3.3867、3.8543、5.0394ms。如果换成那份报告的 ORT FP16基线，倍率约为6.29、6.11、4.78，不能省掉精度口径。

长度不同，算子计算、内存访问和固定调度开销的占比会变，倍率自然不恒定。更重要的是，历史 ORT 开启了 profiling，I/O准备方式也和 TensorRT 不同，因此这些是实现路径的实测耗时比，不是严格排除所有变量后的 TensorRT 纯算法加速比。

P0 45. Benchmark 为什么要固定 warmup、重复次数、输入长度、batch 和 GPU？

第一次运行会混入加载、分配和运行时准备成本，所以要先预热。重复测量是为了看波动，不只挑最快的一次；batch和长度决定了实际工作量，硬件和后台负载则影响资源竞争。

我还会统一是否包含分词、拷贝和同步，并关闭只在一边启用的 profiling。历史测试就存在这些不完全对齐的地方，所以现有数字可以说明本地路径有明显差异，但要作为严格公平的核心性能结论，还需要重新统一计时协议，而不是把倍率再四舍五入得更好看。

P0 46. 为什么最终选择 FP16，而不是 INT8 / INT4？

我选 FP16 主干的混合精度，是因为这条路径在当前硬件上兼顾了速度和数值一致性。最终 embedding 的最小余弦相似度约0.9999966，而更低精度的路径并没有同时带来更好的速度与一致性。INT8在长序列下反而明显更慢，TensorRT INT4还有 MatMulNBits兼容问题，不能说已经部署成功。

实现里也没有把所有算子硬压到 FP16：LayerNorm、Softmax、GELU等敏感区域保留FP32处理。这件事让我更看重实际执行图和误差，而不是只看“几bit”这个标签。

P1 47. 为什么精度更低不代表一定更快？quantize/dequantize、kernel 支持、算子 fallback、launch 开销分别会怎样影响速度？

低精度省下的计算，可能被量化反量化、数据搬运或额外 kernel launch 抵消。如果关键算子没有高效的低精度 kernel，甚至回退到另一种精度或CPU，整段链路反而更慢。batch=1时，固定开销尤其不能忽略。

项目历史记录里，seq=512的 TensorRT INT8约13.47ms，而 FP16路径约5.04ms，就是一个不能只凭位宽预测速度的例子。我不会仅靠这两个数断言具体是哪一个 kernel 拖慢，还需要看 profiler 和执行图才能定位根因。

P1 48. 为什么还要测余弦相似度？>0.999996 在这里证明什么？

加速不能把向量算偏，所以我会用同一批输入，对优化路径与参考路径的输出做余弦一致性检查。最小值超过0.999996，说明这批测试输入的向量方向非常接近参考结果，通过了当次0.999的阈值。

它不证明所有输入永远等价，更不证明检索 Recall或最终行程质量不下降。特别是两个候选分数很接近时，很小的数值差别也可能改变排序。所以 parity是必要的数值检查，下游检索仍要独立验证。

P1 49. 平均延迟和吞吐量分别怎么计算，面试时需要说到什么粒度？

平均延迟就是把有效测量的总耗时除以测量次数；吞吐量则是同一计时范围内处理的样本数除以总秒数。batch=1且串行测量时，吞吐大致就是1000除以平均毫秒数，例如3.3867ms对应约295样本每秒。

这个倒数不是线上并发QPS承诺。服务排队、网络、不同长度和多请求竞争都可能改变结果。面试讲到“怎么算、计了哪些开销、是否串行”就够了，别把局部吞吐量写成整套旅行应用的承载能力。

七 真实HTTP Agent Eval

P0 50. 最新 200 条本机真实 HTTP Agent Eval 测的是什么？为什么 HTTP200=100% 不等于业务成功 100%？

测的是200条请求从本机客户端进入真实 HTTP Agent，再经过检索、模型输出、运行校验和最终业务评分的流程。HTTP200说明接口按协议返回了响应，响应里完全可能是failed，也可能是形式正确但业务不合格的decision。

实际首次成功30条，最终32条，且成功都是澄清响应，130条规划任务没有成功。数据又是AI派生模板集、单客户端本机环境，所以我会把它当成暴露真实链路问题的诊断结果，不会说生产可用率100%，更不会说200条都通过了。

P0 51. 首次业务成功 15%、最终成功 16%、Repair 2/170 应该怎样解释？能不能说“Repair 很有效”？

首次成功30/200是15%；最终成功32/200是16%，只多了2条。第一次失败的170条都进入了修复尝试，恢复2条，所以 Repair Recovery是2/170，约1.18%，不是2/200，也不是最终成功率16%。

这个结果不能支持“Repair很有效”。更诚实的结论是：当前提示词和模型输出能力下，一次修复只能救回极少数任务，还增加了候选生成成本。新增成功也只是澄清类结果，不能包装成规划修复能力明显改善。

P0 52. 为什么最终成功 32/200 不能直接和历史 70 条离线 Final E2E 38.57% 比较？

因为两轮不是同一项实验。70条是固定离线规划对照，38.57%属于历史Base模型分支；200条使用合并模型，走实际HTTP，包含规划、缺信息、冲突和不确定性等类型，生成和接受规则也不同。

所以不能用16%减38.57%就说工程化导致退化，也不能反过来用旧数字说明现在表现更好。真比较，就应该固定同一批任务、模型、证据、输出预算和检查器，把请求入口作为唯一变化，再看结果。

P0 53. 338/370 候选输出不是合法 JSON 暴露了什么主要瓶颈？为什么不能简单归咎于 RAG？

370次候选来自200次首答加170次修复，其中338次不是标准JSON，比例超过91%。这首先说明很多请求连结构化结果都没形成，主要瓶颈暴露在生成和契约适配阶段，业务规则还来不及充分发挥作用。

我会先看原始可见输出、结束原因、token预算和接口适配，再分别验证截断、格式习惯、提示词回显等假设。不能把所有非法JSON都归因于token截断，也不能因为用了RAG就怪检索。检索可能仍有问题，但这份统计没有给出那样的因果证据。

P1 54. P95 E2E=105.195 s 属于什么口径？为什么这个非流式端点不谈 TTFT？

105.195秒是这200条请求的全请求端到端P95，成功和失败都在里面，覆盖客户端实际等待整次请求的时间。它不是模型首字延迟，也不是只有成功任务的P95；后者在记录里约62.235秒，是另一种统计。

这个端点一次返回完整响应，没有token流，因而没有可观测的TTFT，报告里应当是null或不适用。把响应头时间、请求开始时间或整包返回时间叫作首字延迟，都会误导用户对交互速度的判断。

P1 55. 174 次检索走 hybrid-rrf、26 条缺失目的地未检索，说明评测链路和检索指标之间需要守住什么边界？

174次实际检索都记录为同一个 hybrid-rrf索引，26条缺目的地没有检索。这说明我们能区分“执行了哪条检索路径”和“为什么没有执行”，但不能据此算出 Dense-only或Reranker收益。

200条业务评测的分母仍是200，不能把未检索的26条删掉后重新包装成功率。混合检索也只是算法路径名称，不代表相关证据一定找对。要比较排序质量，仍然用固定相关ID的检索集；要评价整条任务，就把未检索、生成失败和业务不合格一起保留。

八 Agent定位与 workflow 设计

新增题56—90。依据当前代码核对，沿用自然面试口吻。记忆确认编辑、统一Token预算等未闭环能力会明确说明；本次仅扩充文档，没有修改业务代码或重跑模型评测。

P0 56. 除了RAG，你在这个项目里实际用了哪些Agent技术？

我会从一条请求怎么跑来讲。用户提交旅行需求后，系统用类型化状态把需求、检索结果和工具结果传下去，LangGraph负责组织检索、天气和酒店查询、规划、校验这些步骤。慢任务由Job管理，前端能看到进度，也能取消；需要确认的行程会停在等待审批的状态。

对话这边还有最近消息、历史摘要和长期用户偏好。外围配了有限重试、总执行预算、检查点和来源记录。这些能力解决的是具体问题，比如刷新后任务不能丢、旧偏好不能压过本次要求。我没有做多个角色互相聊天的多Agent系统，也没有让模型自由执行任意工具。现在更准确的说法是受控工作流式Agent。

P0 57. 固定流程也能叫Agent吗？和普通聊天机器人区别在哪？

我不会在名字上争论太久。这个项目的模型输出要进入一个可执行、可校验的业务流程：它需要外部资料，能使用服务端工具，输出结构化行程，而且有明确的任务状态和失败处理。普通问答到一句回答就结束了，这里还要判断行程是否符合请求、是否需要确认。

不过它的自主性确实有限。主要节点顺序由代码确定，不能说已经实现了通用自主Agent。面试里我更愿意把“模型能决定什么、程序限制什么”讲清楚，这比给它贴一个很高级的等级更有用。

P0 58. 为什么选LangGraph？不用它，普通函数串起来不行吗？

完全可以。只有四个线性步骤时，普通函数甚至更直观。我保留LangGraph，是因为检索、工具、生成和校验有各自的状态更新，也需要从已完成的读取节点继续执行；用显式节点和边能比较容易看出某个阶段接收什么、产出什么。

但LangGraph没有替我解决数据库事务、用户鉴权和审批并发。我把这些留在Job服务层，而不是再让图维护一套对外任务状态。否则遇到图说结束、数据库说等待审批的时候，会很难解释到底谁对。

P1 59. 你用了ReAct、Plan-and-Execute或者多Agent吗？

当前主链路没有做开放式ReAct循环，也没有拆成规划师、审核员等多个模型角色。它是retrieve、tools、plan、validate的受控流程。新HTTP评测入口有最多两个候选的修复机制，但这不能等同于一个能反复思考、自由选工具的ReAct Agent。

我暂时不加多Agent，是因为目前瓶颈已经很明确：小模型连复杂行程的合法结构都不够稳定。这时多加一个模型角色，很可能增加调用成本和错误传播。以后如果确实有独立任务可以并行，比如不同交通方案比选，我会先做单模块对照，再决定是否拆成多个Agent。

九 记忆与上下文工程

P0 60. 你的记忆系统具体怎么设计？短期记忆和长期记忆分别存什么？

我没有把所有聊天记录都叫长期记忆。最近几轮消息负责保留眼前的上下文；更早的对话压成摘要，保留目标、限制和待确认问题；长期记忆单独存用户比较稳定的偏好、约束或事实，例如更喜欢慢节奏旅行。

代码里分别有ChatSummary和UserMemory，摘要带已覆盖消息数量，长期记忆有用户归属、类别、置信度和过期时间。聊天时再组装成PromptMemoryContext。这里要注意：当前明确接入的是聊天链路，不能因为有记忆模块，就声称每一个结构化行程入口都自动读了长期记忆。

P0 61. 为什么不把全部聊天历史直接塞进Prompt？摘要怎么生成？

全部塞进去很省开发时间，但对话长了之后，费用和延迟会增加，旧需求也会干扰这次请求。所以实现里会留出最近消息窗口，把更早且尚未覆盖的部分交给摘要器，再记录covered_message_count，下次只处理新增的旧消息。

摘要不是无损压缩。用户先说去杭州，后来改成南京，摘要如果漏掉这个转折，就会制造一个看起来很确定的错误。我会保留原始消息作为追查依据，不把摘要当作用户原话；真正的天数、预算等执行参数还是从当前结构化请求来。

P0 62. 长期记忆怎么抽取？模型能随便修改用户画像吗？

抽取器返回的是受schema约束的动作，包括ADD、UPDATE、DELETE和NOOP，类别限定在偏好、约束和事实。对话抽取时，代码只把新出现的用户消息送进去；提示词要求临时请求、模型自己的回答和猜测返回NOOP。这样至少先避免把助手编出来的内容又存成用户事实。

存储层还会检查置信度、更新目标是否存在等条件。不过模型给的置信度不是经过校准的概率，提示词也不是安全证明。当前确认标志、修订号和证据字段已有部分补丁，但确认编辑接口和完整写入链路还没有交付，我不会把它说成已经成熟的用户画像管理系统。

P0 63. 旧记忆和用户这次的要求冲突，听谁的？

我会优先听这次明确的要求。比如以前记录了“喜欢爬山”，这次用户说“带老人出门，不安排登山”，就不能为了个性化继续推荐山路。聊天提示词已经写了当前要求优先，历史摘要和记忆也被放在参考数据里，没有提升成系统指令。

但目前注入的长期记忆主要还是内容字符串，确认状态和证据没有完整传给模型。因此我只能说优先级有提示约束，不能说所有冲突都被程序可靠消解。更稳的下一步是保留来源和确认状态，在拼上下文前先做明确冲突的过滤，并且不因为一次临时要求就覆盖长期偏好。

P1 64. 记忆是向量检索吗？用了Mem0或者专门的记忆框架吗？

当前不是。用户记忆主要由自己的MemoryStore和MemoryService管理，按用户取出候选，结合确认状态、置信度、更新时间和TTL筛选，再限制条数。这里没有依据当前问题做一轮独立的记忆Embedding相似度检索，也不能把旅游知识库的BGE检索算作记忆向量检索。

对于一个用户只有少量稳定偏好的场景，这样做比较容易解释和删除。如果以后积累到大量历史事件，才值得引入语义检索；那时还要处理相似但相互冲突的偏好，不能只按向量分数高低来决定哪条是真的。

P0 65. 如何避免同一段话被反复抽取，产生重复记忆？

对话链路给用户消息生成来源哈希，里面包含用户、会话、消息位置和文本，再记录哪些来源已经处理过。这样同一轮任务重复触发时，不会一直把同一句话当新证据；把位置也算进去，是为了区分用户后来再次表达的相同意见。

这里仍有一个值得继续加固的窗口：代码先登记来源已处理，再逐条应用抽取结果。中间崩溃可能导致登记成功、记忆没有完整写入；进程内锁也不能代表跨进程一致性。我要做完整保证，会把处理标记和记忆变更放进同一个事务，或者设计可重试的处理状态。这是代码审查能指出的风险，不是我虚构的一次线上事故。

P0 66. 用户能查看、删除记忆吗？你怎样处理隐私和过期？

已经有登录用户查看自己记忆、删除指定记忆和触发刷新这些接口。用户身份来自认证上下文，删除时同时检查归属，不能靠传一个user_id去操作别人的记录。进入Prompt前还会过滤过期和不满足置信度要求的记忆。

删除一条偏好不等于清除了所有关联信息，原始聊天和摘要是不一样的数据。所以我不会把当前删除接口描述成完整的隐私擦除能力。还有，TTL过滤解决的是“不再使用”，是否物理清理、备份如何过期，是另一层工作。当前用户确认编辑和彻底的删除防复活流程仍要补齐验证。

P1 67. 上下文长度怎么控制？你做到了精确Token预算吗？

目前有几道粗粒度限制：最近消息数量、长期记忆条数，以及聊天Prompt里检索内容和摘要的字符截断。它能避免无限增长，但字符数不等于Token数，不同语言和模型的分词都可能不同。最近消息里如果有很长的一条，也可能把总上下文撑大。

因此我不会说已经有完整的Token预算调度。我要补的是统一组装入口，先给系统指令、当前请求和输出预留空间，再按重要性放入近期对话、摘要和检索证据，最后用当前模型的Tokenizer计数。不能每个模块觉得自己只占一点，合起来却超了。

P1 68. 记忆抽取为什么放后台？后台任务失败或服务重启怎么办？

记忆更新不应该拖住用户看到本轮回答，所以现在通过线程池异步处理，同一会话的重复排队也做了限制。抽取失败会记日志，主回答已经完成，不会因为画像更新失败又被改成失败响应。

这个实现是进程内后台任务，不是持久消息队列。重启时还没执行的工作可能丢失，也不能保证已经抽取的结果一定落库。当前适合做非关键增强；如果以后记忆写入成为必须完成的业务，我会加持久任务和幂等消费，而不是把线程池说成可靠队列。

P0 69. 你怎么证明记忆确实有用，而不是只是存了一张表？

至少要证明三件具体的事：换一个会话还能正确使用稳定偏好；本次要求冲突时旧记忆不会抢优先级；另一个用户完全读不到这条信息。删除、过期和重复抽取也要能用固定输入复现。

我现在能指到聊天注入和存储筛选的代码，但还没有一组可信的对照结果可以说“记忆让规划成功率提升了多少”。如果面试官继续问收益，我会直接承认这一点。接下来应该拿同一批多轮任务比较开启和关闭记忆，分别统计约束遗漏和错误沿用，而不是只展示数据库里多了几行记录。

十 工具、校验与执行边界

P0 70. 工具是谁决定调用的？做了Function Calling吗？

主要由服务端流程决定调用天气和酒店POI工具，模型拿到筛选后的工具结果再规划。工具入口有固定名称和参数schema，没有根据模型生成的字符串动态找函数，也不接受任意URL。这和模型自己返回tool_calls、运行器再执行的Function Calling不是一回事。

我这样做是为了先把调用范围和故障路径控制住。等模型在工具选择和参数生成上足够稳定，可以让它提出调用意图，但执行前仍必须经过白名单、参数校验和预算检查。模型建议调用，不代表模型拥有执行权限。

P0 71. 工具参数校验和结果校验，你分别做了什么？

参数侧有固定的天气和酒店输入模型，会限制目的地、日期、数量等字段。结果侧也要检查上游业务状态、数据结构和字段范围，最后归一成带available和错误信息的结果。只拿到HTTP200，还不能说明酒店列表或天气日期是有效的。

有个很容易说错的地方：酒店POI只能说明附近有酒店，不能证明实时有房，更没有给出可成交的房价。所以工具结果把价格和库存的未知状态单独表达。面试里我会讲这个例子，因为它说明结果校验要懂业务含义，光用Pydantic检查类型还不够。

P1 72. 外部工具挂了怎么办？缓存会不会给用户过时信息？

天气和酒店查询是尽力而为的辅助数据，失败会返回不可用并产生降级提示，规划Prompt只接收available的数据。工具还用了TTL缓存，并保留原始获取时间，缓存命中不会伪装成刚刚查到的数据。天气和酒店的有效期不同，不能套一个统一时长。

如果天气预报根本不覆盖出行日期，就应该明确未知，而不是拿今天的天气当下个月的天气。降级后能继续生成参考草案，但不能暗示外部事实已经确认。缓存解决性能问题，不会自动解决事实的新鲜度。

P0 73. 规则和LLM怎么分工？Pydantic通过了就算成功吗？

LLM负责把用户要求和证据组织成行程，规则负责能确定判断的部分，例如天数、活动顺序、时间重叠、人数费用和预算口径。Pydantic先保证结构能被程序消费，后面的业务校验再看这个结构是不是符合请求。

比如一个方案JSON完全合法，却把三天写成两天，或者单人价格忘了乘人数，都应该继续被拦截或标注。反过来，字段类型正确、费用加总也正确，并不能证明景点真的开放。我会把结构正确、约束正确和事实可信分开说。

P0 74. 检索到了source_id，就能说模型没有幻觉吗？

不能。source_id存在，只证明引用到了返回的某条资料，不证明活动里的每句话都被那条资料支持。项目里就需要把“找到来源”与“事实已验证”分开，避免给用户一种已经核验过票价和开放时间的错觉。

当前会处理未知引用，并把证据不足的内容标成待确认。更细的事实核验还需要把断言和证据逐项对齐；没有做这一步，就不能因为引用列表不为空而把整份行程称为无幻觉。

P0 75. 重试怎么设计？为什么不失败就一直重试？

我把网络重试和模型输出修复分开。网络侧只对策略允许的瞬时错误重试，比如连接失败、超时或限流，配合最大次数、指数退避和抖动；服务端有Retry-After时也不能无视它提前重试。参数本来就错了，多调几次不会变对。

模型修复则是在候选不符合要求时再生成，属于另一个成本来源。新HTTP评测入口最多两个候选，而不是无限循环。真实结果只从170次首次失败里救回2次，所以我不能说加重试就让小模型可靠了；很多时候应该尽早明确失败，让用户调整需求。

P1 76. 每个工具都重试三次，会不会把整个请求拖垮？

会，所以主旅行工作流还有共享的ExecutionBudget，用单调时钟限制总时长，同时限制外部调用尝试数。节点开始和同步外部调用前会检查预算，超时配置也会参考剩余时间。这样不能让RAG、天气和模型各自花掉一整份请求预算。

这个预算协作式的，不会强杀一个已经卡住的线程，底层I/O仍然必须有自己的超时。而且它当前明确覆盖的是这条同步工作流，不能顺口扩展成所有异步聊天和后台记忆任务都共享同一个预算。

十一 状态、审批与恢复

P0 77. Agent的状态放在哪里？怎么避免前端和后端状态不一致？

我分成两层。TripState保存本次图执行的请求、来源、工具上下文和方案；TripPlanJob保存对外可见的任务状态、结果、错误、进度以及审批信息。前端展示以Job为准，不能根据最后一个token或者某个节点完成事件自行宣布整个任务成功。

例如规划节点完成后，还可能在校验，或者等待本人审批。状态模型区分queued、running、awaiting_approval和几个终态。MySQL提供持久记录，但普通任务也有内存或待持久化状态；我不会把所有路径都描述成数据库强一致。

P0 78. 人工审批是怎么做成闭环的？点确认后发生什么？

需要审批的任务先生成并校验草案，再持久化为awaiting_approval。用户决定要绑定任务归属和草案版本；服务端在事务里锁住对应行，确认版本和当前状态，再接受批准或拒绝。重复提交相同决定返回已有结果，不能重复执行。

当前这个最小闭环里，批准是把已经校验的草案确认为结果，拒绝则结束该任务。它不会自动订酒店、扣款，也不是审批后偷偷再让模型生成一份不同的行程。修改需求应该重新提交、重新生成，拿到新版本后再决定。

P0 79. 为什么审批要带版本？两个页面同时点不同按钮怎么办？

用户确认的是眼前这一版内容，而不是抽象的任务编号。所以草案内容生成哈希版本，决定必须带上这个版本。如果另一个页面已经处理了决定，后来的冲突请求就不能再把结果翻过去。

数据库的行锁和状态检查把并发决定收在一个事务里。相同决定可以幂等返回，不同决定应报冲突。仅在前端禁用按钮没有这个保证，因为用户可以开两个标签页，也可能在网络超时后重发请求。

P0 80. 数据库不可用时，审批能否先在内存里成功，之后再补写？

审批这条路径我倾向于直接失败关闭。用户一旦看到确认成功，就会合理地认为自己的决定已经保存。如果实际只在内存里，重启又回到等待确认，这个承诺就破了。

当前需要审批的任务要求有持久仓库，审批决定也走数据库事务。普通参考型任务可以明确显示内存降级，但不能把这种容错方式套到审批承诺上。可用性不是唯一目标，有些操作宁可暂时不可用，也不能假装完成。

P0 81. 你的断点恢复是不是LangGraph原生Checkpoint？

不是。当前图是直接compile的，检查点由项目自己的TripCheckpoint和Job服务管理。只在retrieve或tools这种已完成的读取节点后保存请求指纹、版本、时间以及结果，恢复时决定从后一个节点继续。

这样做范围比较小，也容易说明副作用边界。它不是把完整Python执行栈序列化，更不能恢复到模型生成第几个token。以后接原生checkpointer当然可以讨论，但现在面试要按代码讲，不能因为用了LangGraph，就默认它所有持久化能力都已经接好了。

P1 82. 恢复之前为什么还要检查请求指纹、版本和TTL?

因为检查点可能已经不适合当前任务。用户改了目的地，旧检索结果就不能复用；状态schema或工作流版本变了，旧数据的解释也可能不同。代码会比较请求指纹、两个版本和保存时间，失效就重新跑完整流程。

TTL只是一个粗边界，不代表所有工具事实在这段时间里都有效。例如天气还有自己的日期覆盖范围。当前恢复也不能宣称对每个上游数据版本都做了完全一致的快照校验，这是进一步强化恢复语义时要补的部分。

P0 83. 用户点击取消时，模型正好生成完，会出现什么竞态?

这是很实际的一个问题。如果先发completed，再检查取消标志，用户就可能同时看到完成和取消。项目里对这个路径做过收敛：由任务层在同一把锁下决定最终状态和对应终态事件，不让工作流节点自行发布整个任务的成功。

不过取消仍是协作式的。已经发给外部模型的请求，不保证立刻停止计算或计费；我们能保证的是取消后不再继续接受正常完成结果，并在检查点停止后续工作。面试时我不会把“按钮立刻变成已取消”说成GPU推理已经被强制中断。

P0 84. 提交幂等、重试和失败恢复有什么区别？能做到Exactly Once吗?

提交幂等解决的是同一个用户因为双击或网络重发，不应该创建两份相同任务；重试是失败后发起新的尝试；恢复是在新尝试里复用仍有效的读取结果。代码保留父任务关系，也把幂等键和用户归属关联起来，避免跨用户复用。

我不会承诺通用的Exactly Once。数据库内的审批决定可以用事务限制，但如果将来真的接支付或预订，外部服务是否已经执行，可能在超时后变成未知。那需要上游幂等键、状态查询和补偿机制，单靠一个本地任务ID解决不了。

十二 安全、可观测性与面试深挖

P0 85. SSE展示的是模型思维链吗？断线以后怎么恢复？

不是思维链。展示的是程序可观察到的阶段，比如开始检索、找到多少来源、工具降级、开始生成以及最终状态。模型内部推理既没有必要展示，也不应该被当作可靠的执行日志。

浏览器断线不代表后台任务失败。客户端应该凭任务ID重新取Job状态和已有进度，确认它是运行中、等待审批还是已结束。现有记录是有限历史，我不会承诺无限事件重放或每个token都能原样补回；断线后能恢复任务结果，和完整恢复流内容是两件事。

P0 86. 日志、审计记录和回放有什么区别？现在做到了哪一步？

日志主要帮我定位请求在哪个阶段出问题，任务记录保存用户能理解的结果和状态，ExecutionManifest再记录工作流版本、模型标识、来源ID和恢复关系。通过request_id与job_id，可以把这些信息关联起来，而不是在一大段文字里猜是哪一次请求。

现在保留的是轻量的过程追踪和只读历史，不是合规审计平台，也不是确定性重放系统。即使模型名和输入一样，服务版本、随机性或者检索索引变了，输出也可能不同。要声称精确重放，必须再固定相应的输入、索引、参数和代码版本。

P0 87. 检索文档或长期记忆里出现“忽略之前指令”，你怎么防？

我把这些内容当外部数据，而不是新的系统指令。聊天Prompt里会明确参考JSON不是本次指令，当前用户请求单独放置；工具侧也不让模型提供任意URL或动态函数名。这样即使一段资料带有恶意文字，也不应该直接获得执行权限。

但只写一句“忽略恶意指令”不等于完成了防护，结构化规划提示词与聊天入口的隔离程度也要分别检查。更可靠的底线是权限和参数在代码里校验，外部数据不可以修改身份、工具范围或审批状态。当前没有证据支持“已经彻底解决Prompt Injection”这种说法。

P0 88. 匿名用户也能建任务，怎么防别人猜ID拿到行程？

资源ID不是权限。登录用户绑定自己的身份，匿名用户通过服务端的owner机制关联会话，读任务、看进度、取消和审批都要检查归属。别人即使拿到一个job_id，也不应该因此获得访问权限；外部资源查询尽量统一返回404，避免暴露它是否存在。

我会专门拿两个身份做交叉访问，而不只测本人访问成功。还要检查SSE和重试接口，因为它们经常是主查询加了权限、旁路接口却漏掉的地方。日志也不能为了排障把匿名凭证或Token直接打印出来。

P1 89. 哪些Agent能力还没做？下一步最值得补什么？

我会先补记忆闭环，而不是先换模型：确认编辑的版本检查、来源证据回填、删除后不被旧抽取重新写回，以及所有上下文入口的统一预算。接着把已有状态与工具失败路径的验证补完整，尤其是多进程竞争和持久化失败。

开放式工具选择、多Agent协作、自动下单、任意节点原生恢复都不在当前已完成能力里。模型和Embedding也先保持不变，这样后续改动才比较容易定位效果来自哪里。等受控流程能稳定完成任务，再逐步放开模型的决定范围。

P0 90. 面试最后让你讲一个最能体现Agent工程能力的案例，你讲什么？

我会讲取消和完成的竞争。表面上它只是一个停止按钮，真正做进去才发现，模型结果回来、进度事件发布和用户取消可能同时发生。只在界面上改状态没有用，必须让任务层统一决定终态，不然用户会看到“已取消”旁边又出现一份成功结果。

然后我会顺着讲审批：用户确认的必须是某一版草案，决定必须属于本人并且能持久保存。我觉得这两个例子比多加一个Agent角色更能说明我做过什么。模型输出只是中间一步，系统最后对用户承诺的状态，才是必须认真守住的东西。

十三 指标不理想时，怎么经得住追问

P0 91. HTTP业务成功率才16%，而且规划0/130，你这个项目到底能不能用？

按这轮评测结果，我不能说它已经能稳定完成旅行规划。200条请求最终通过32条，而且这32条都是澄清响应，130条规划没有成功。所以16%是整批混合任务的业务成功率，不能拿来当行程规划成功率。HTTP全部返回200，只说明请求得到了响应，不代表用户拿到了合格行程。

这轮测试的价值，是把演示时不容易看见的问题暴露出来了。检查候选输出后，大量请求在标准JSON解析阶段就失败，生成和结构化适配是优先排查的位置。但这还不足以把所有失败都归因于模型小，或者说只改格式就能解决。我的下一步会先拿固定小样本分清截断、格式错误和业务约束错误，逐类修复后回到同一批任务验证。目前工程流程可以展示，规划效果还没有达标，这两点我会一起说。

P0 92. 338/370个候选不是合法JSON，这么基础的问题为什么还会出现？

这个比例确实很高，约91.4%。370是包含修复尝试的候选数，不是370个独立用户请求。我不会用“系统有结构化输出模块”来回避这个结果。有schema只代表程序知道想要什么，并不保证当前模型和调用路径每次都能输出符合要求的内容。

我会先保留原始输出，结合结束原因和输出长度看问题发生在哪里：是截断了，是夹带解释文字，还是字段和括号本来就不对。不同原因要分开处理。如果仅仅靠宽松解析把它抢救出来，也要单独记录，不能说它原本就是标准JSON合格。现在证据支持的是结构化输出链路有明显短板，还不能支持某一个原因解释全部338次失败。

P0 93. 修复只救回2/170，为什么还保留Repair？你做的是无效优化吗？

这次结果说明当前修复策略收益很弱，我接受这个结论。首次失败170条，最终只救回2条，恢复率约1.18%；整体成功率也只是从15%到16%。不能因为多了两个成功，就把Repair包装成可靠性的大幅提升。

我会保留它作为可对照的实验路径，但是否默认启用，要看它带来的额外调用和等待是否值得。下一步应先按错误类型判断哪些值得修复，比如局部格式问题和复杂约束失败可能需要不同处理；对于基本没有收益的类型，直接明确失败更合适。现在还没有完成这轮分类验证，所以我不会说已经找到了高效修复方案。

P0 94. P95要105秒，用户为什么要等？做了SSE就能解决吗？

不能，SSE只能让等待过程可见，没有把105秒变短。这个数字是200条请求的客户端全请求P95，包含失败和修复开销。只报成功请求约62秒的P95，会把最慢的一部分体验藏起来，而且62秒也谈不上快。

如果继续优化，我会先拆检索、模型生成、排队和修复耗时，找真正占大头的部分。当前修复收益很低，是值得优先检查的额外开销，但没有阶段计时不能直接认定它贡献了多少延迟。异步任务、取消和进度展示是已有的交互保障；实际时延仍要另做优化。这轮还是本机实验，也不能拿它推算生产环境的并发承载能力。

P0 95. Reranker几乎没提升，反而慢七倍，你为什么要加它？

加它做对照，是为了判断它在这个项目里值不值得用，不是预设它一定更好。150条同集查询上，MRR从0.977778到0.984444，Recall@5仍是0.990667；实际只有两条查询从第二名提到第一名。与此同时，检索进程内P95从44.13毫秒增加到313.10毫秒，约7.1倍。这个代价不小。

我的判断是，这组样本上的收益不够支持默认开启。Dense本身接近满分，测试也可能缺少真正困难的排序场景。后续可以补困难查询，再研究是否只在需要时重排。但“这批数据不划算”和“Reranker没有价值”是不同结论，我只对这次实验能说明的范围负责。

P0 96. 70条离线任务最终也只有38.57%，合并模型部分指标还下降，优化到底有没有用？

有局部改善，但远没有达到稳定可用。Base分支的最终通过数从18/70到27/70，也就是25.71%到38.57%。这个改进是真实的，不过还剩43条没有通过，不能只强调相对提升，省略最终水平。

合并模型要单独看：那一支的约束宏平均从60%降到55.67%，不能拿Base的上升替它背书。同一套提示调整没有在不同模型上全面奏效，说明优化不能只看一个总分。我接下来会对照哪些任务新通过、哪些新失败，再按约束类型定位变化。现在不能把最好看的模型分支和最好看的指标拼成一份成绩，更不能把提示修改说成一次新的模型训练收益。

P0 97. Faithfulness有0.866，但只评了31/50条；Relevancy的0.685又说明什么？

Faithfulness这组最容易误导人的地方，是只说均值，不说覆盖率。约0.866是31条有效评分的均值，另外19条没有有效评分，覆盖率只有62%。它们不能直接算正确，也不能不分析就算错误；但省略它们，会让人误以为整批50条都有同样的可信度。

Relevancy约0.685来自项目的自定义评分口径，不是68.5%的答案正确率。相关性和忠实性也各自回答不同的问题，一个回答可以很贴题，却编造事实。我会报告评分方式和有效样本数，再查清无法评分的原因。没有这些前提，就不把两个分数拿去证明整个系统质量已经合格。

P0 98. 检索Recall接近98%，为什么端到端规划还这么差？这个高分是不是水分很大？

首先，这不是同一批数据、同一条链路的两个分数。351条的Recall@5约0.9801来自历史BM25回归，查询和语料的关联比较强；200条HTTP任务测试的是从输入到最终业务结果的整条链路。不能据此直接建立“检索98%，生成就应该接近98%”的关系。

我认可这套检索集有局限：它更适合检查固定资料能不能被找回来，不能充分代表真实用户的开放表达和复杂条件。即使资料找对了，模型仍可能生成非法JSON或违反行程约束。所以我会保留它做回归，同时补更贴近使用场景、独立审查的查询；不会把这个数字叫BGE效果，也不会靠它掩盖端到端失败。

P0 99. TensorRT快4.87—7.35倍，计时条件却不一致，这个加速数字还能讲吗？

能讲，但必须把限定条件和数字放在一起。原始CSV里的倍率对应ORT CUDA FP32基线，不是FP16基线，也不是完整Agent端到端加速。我核对过不同序列长度下的原始耗时，算术上能对应，但两条路径的profiling和I/O方式没有完全对齐。

所以它能支持的说法是：历史本地测试配置下，优化路径测得了更低耗时。它不能严谨证明所有收益都来自TensorRT本身。如果要做公平性能结论，我需要统一输入输出传输、同步、预热和profiling设置再测。在补齐之前，我宁可多解释一句实验限制，也不会说整个旅行应用快了七倍。

P0 100. 这么多结果不理想，你为什么还把项目写进简历？哪些值得重点讲？

因为我能讲清楚自己实现了什么，也能拿出结果说明哪些地方还没做好。但这并不意味着失败指标本身就是成绩。我不会把这个项目写成成熟上线、能稳定完成复杂旅行规划的产品；目前更适合展示 workflow 编排、任务状态、来源校验和可复现排障这些工程工作。

面试展开时，我会重点讲取消与完成的竞争、审批为什么绑定草案版本，以及为什么没有因为用了 Reranker 就默认开启。它们体现的是我怎么判断问题和做取舍。被问到模型效果，我就给出实际数字，不换分母、不只挑成功案例。接下来真正需要做的，是让规划在固定任务上稳定通过；解释得再清楚，也不能代替把效果做好。

最后三十秒怎么收尾

我能拿出固定样本、原始输出、检查器和失败记录，说明每个指标来自哪里。检索命中、JSON合法、业务成功和接口成功，我会分开讲。

项目也有不理想的结果：合并小模型在复杂规划上仍有明显短板，Reranker的收益很小，历史加速对照也有计时限制。我更愿意把这些问题讲透，说明怎么定位、怎么验证，而不是把它包装成一个已经成熟上线的旅行产品。

核对依据

以下为项目内可定位的材料。本次填写没有重跑模型评测，也没有把中断中的记忆补丁算成已验证功能。

题1—26及40—41

docs/features/mindtrip-project-experiment-report.md; experiments/evaluation-release-20260907/retrieval-comparison-report.json

题9—13及20—21

Alzhilv/rag_service/evaluate.py; Alzhilv/rag_service/retriever.py

题27—39

Alzhilv/backend/app/agent/trip_workflow.py、trip_validation.py、trip_agent.py; schemas/trip.py; services/trip_jobs.py、trip_job_persistence.py

题42—49

experiments/local-20260828-1852-bgepipe/benchmark/bge-benchmark-results.csv;
docs/features/mindtrip-project-experiment-report.md 第8节及性能口径补充

题50—55

experiments/http-agent-200-20260908/metrics.json; docs/features/http-agent-evaluation.md

题8工程案例

取消与完成竞态、持久审批重启恢复、隔离MySQL验证、H5交互记录；对应test_trip_jobs.py、test_trip_job_approval.py、test_trip_job_closeout.py。

新增题56—76

Alzhilv/backend/app/ 下：services/memory_service.py、memory_store.py; schemas/memory.py; api/memories.py、assistant_chat.py; agent/trip_workflow.py、trip_tools.py; utils/retry.py、execution_budget.py。

新增题77—90

Alzhilv/backend/app/ 下：schemas/trip_jobs.py; services/trip_jobs.py、trip_job_persistence.py; api/trip_jobs.py; progress.py。答案区分当前实现、局部未验证补丁和后续设计；未将进程内锁描述为分布式保证。